

5

TEXT PROCESSING TECHNIQUES AND TRADITIONS (OR: WHY THE HISTORY OF COMPUTING MATTERS TO DH)

John W. Maxwell

Introduction

Why should digital humanists care about the history of technology, of computing, or of the digital? Does computing merely provide a toolset that we employ in order to achieve predefined ends? Are we to take digital technologies at face value, as given to us by our predecessors and our disciplines? Or is there—should there be—a humanistic interest in digital infrastructure itself? In this chapter I want to make the case that attending to the cultural histories of our computing environments is not merely illuminating for our tool use but is also a valuable application of humanities scholarship. I argue that we draw upon long traditions of computing which shape the way we think and act with our digital tools and which contribute to a cultural background that we inherit when we use digital media—whether we're aware of it or not. Gaining some critical awareness of those traditions through historical perspective is essential to humanistic study on many levels.

The traditions I refer to here are not easily reducible to a superficial history of computing, rooted in Cold War and military-industrial complex narratives, such as recounted by historians like Paul Edwards. Rather, I want to identify some features of a tradition that is contiguous with much older intellectual currents in the West, and especially with the history of print and of the book. Rather than positioning digital media as something that represents a challenge or break from print culture, I see it as largely made of the same stuff—on “material” grounds, as well as more theoretical ones. Rather than technological change introducing so many breaks with the past, I make a case for overlapping conceptual frameworks that build upon and challenge each other while coexisting. Such an assertion requires that we attend to the details of the history and the perspectives that illuminate it.

Traditions, of course, are not monolithic—the stories I highlight in this chapter are among many that can be picked from the details of history and historiography.

My aim here is to situate digital technologies and tools as *textual* traditions, to draw out the continuities in digital culture and print—or better, *typographic*—culture, and to show how the general “toolkit” for text processing on computers represents a complex inheritance that reaches back decades and even hundreds of years. My aim is not to debunk mythology, nor to expose the hidden secrets buried within these stories. Rather, with Donna J. Haraway,

I want to know how to inhabit histories and stories rather than deny them. I want to know how critically to live both inherited and novel kinships, in a spirit neither of condemnation nor celebration. I want to know how to help build ongoing stories rather than histories that end.

(1)

Digital humanists’ relationships with technology and techniques are ongoing stories, and my hope is that we can critically live with both what we inherit and what we may yet dream up.

Finding positions

“Unix [...] is not so much a product as it is a painstakingly compiled oral history of the hacker subculture.”

—Neal Stephenson

I wince at invoking the “hacker subculture” in the era of Trump, as we have seen horrors committed in its name already. And yet, in the third decade of the digital age, this is certainly part of our daily life online; it is utterly shot through the infrastructure of contemporary culture. Hacker culture, as with the computing history with which it has been entangled, is also complex. It is older, richer, and bigger than the now-popular image of angry young libertarians might suggest. Polarizing discourses here are easy, because the digital world is a politically charged space. The not-unrelated charge that the “digital” and the “humanities” are in fact antithetical is one which comes up perennially; it is based, I suggest, on a particular kind of reading of the history and traditions of computing. This is one reason why I think we need to attend to the history and traditions of computing with greater care.

For instance, a superficial reading of Tara McPherson’s most excellent and insightful essay, “Why are the Digital Humanities So White?” might lead one to identify the Unix operating system with the worst of contemporary political alignments. I believe a more careful reading of McPherson reveals a more complex picture—more on that to follow—but Neal Stephenson’s point about Unix as oral history is an important one. There are foundational ideas here. Systems are not simply designed by engineers and implemented in software that is then thrust upon the rest of us. Rather, they are evolved over many years and even generations of both developers and users. Such ideas and manifestations bear examination, exploration,

critique, and appreciation, especially since we tend to live and work in environments that derive from that evolution.

In “Why are the Digital Humanities So White?” McPherson poses the question thus:

Might we ask whether there is not something *particular to the very forms* [emphasis in original] of electronic culture that seems to encourage just such a movement, a movement that partitions race off from the specificity of media forms?
(143)

and, further on,

It would be foolish of us to believe that these operating systems—in this paper best categorized by UNIX and its many close siblings—do not at least partially overdetermine the very critiques we imagine that we are performing today.
(152)

It is an excellent question, and the latter is an incisive point, especially as much of the digital humanities sits comfortably upon platforms and foundations that derive directly from Unix and its contemporaries. This is, I believe, exactly the kind of question that we need to ask ourselves as we inherit three or more decades of accrued practice and tools around computer text processing—not to mention the castles and cathedrals of more recent tools and techniques that have been built upon that same foundation.

But why stop there? Why assume that the computing technologies of the '60s and '70s and the attendant rise of institutional computing holds the key when, as I will argue, these are very much part of vastly longer traditions in Western culture: in print culture, to say nothing of mathematics, capitalism, and liberal democracy? I am not sure there is much to be gained in playing print against computing or vice versa. The traditions are long and the threads very much interwoven. My agenda here is to tease out some of those historical threads by making reference to some specific practices and forms and to draw out the technological relationship between practice and imagination.

1. *The form of the book*

The story of the long road to the digital that I want to tell begins with *typography*, specifically at the point where letters become something made by machines rather than (or at least in addition to) pens. Our typographic tradition is vast: from the advent of the alphabet through scribal culture, the innovation of moveable type and the age of print, through industrialization, automation, and ultimately computerization. Typography is a lens through which we can focus on the mechanization of the word (to borrow a concept from McLuhan and Ong), a process centuries and millennia old and at which we are still actively working.

The conventional starting place for discussions of typography is with Johannes Gutenberg, although Gutenberg is very clearly in the midst of much longer-running traditions of typography and indeed printing, in both graphic and material senses. Gutenberg, whatever his personal contributions may be,¹ stands for the historical point at which printers learned how to mass-produce—that is, to industrialize—documents. His invention is not the printing press, nor even moveable type; more properly, Gutenberg is credited with developing the mechanism for *mass-producing* the *interchangeable parts* (i.e., letters) of the alphabet out of a lead alloy, at scale.

It was and is highly technical: the forms of individual letters, cut in relief into pieces of soft steel (see Smeijers), the steel then hardened in a forge, punched into softer copper to make an impression of the type, which is clamped into a clever interlocking two-piece mould, into which is poured an alloy of lead, tin, and antimony. The resulting individual “sorts”—all of identical dimensions—were kept in two boxes, the “upper case” and the “lower case” and picked out one by one by nimble fingers and arranged, letter by letter and line by line, upside down and backwards.

The whole thing was then clamped and locked into a single “forme,” rolled with ink and then pressed into sheet after sheet of paper. Pages hung, dried, folded, and bound, done for centuries *all by hand*. This process was incredibly meticulous, even from the very beginning: Gutenberg’s famous 42-line Bible, as early as the 1450s, is a marvel of precision and exactitude—as is much of the wealth of printed works from the decades that followed. This is the scaling up of handcraft to industrial levels, the model for the Industrial Revolution that followed.

Print is a mechanized process, but it is very labour intensive. Automation came very slowly and not in earnest until the 19th century. In the meantime, typographers formed some of the first trade guilds and one of the first—and strongest—trade unions. It is a trade and craft that organized not just labour but knowledge, passed from master to apprentice (almost always men)² over years, decades, lifetimes.

Such a concentration of power and knowledge in labour was perhaps doomed to collide with industrial capitalism’s appetite for, well, everything. As such, with the desire for scale and speed came automation. In the 19th century, steam-powered printing presses and paper-handling machines sped up production enormously. Bindery presses appeared shortly after, and the handcraft aspects of printing declined by degrees (see Lundblad). Typesetting itself, however, remained a craft performed by highly trained hands until very recently. The difficulty of mechanizing and automating the assembly of words and lines, especially to the high aesthetic standards of the tradition, required serious engineering efforts.

In the 1880s, two almost simultaneous inventions ushered in the age of “hot metal” type and machine composition: Ottmar Mergenthaler’s *Linotype* and Tolbert Lanston’s *Monotype*. Both were machines directed from compositor’s keyboards (like a typewriter but a parallel development as opposed to a derivative). Both machines automated the casting of type metal and achieved something like an order of magnitude improvement in speed over hand-setting. The names of the two machines indicate the key difference: The Monotype machine was popular in book printing and cast a series of single letters which were then assembled in a forme, like

in traditional letterpress. The Linotype machine, which revolutionized newspaper production, cast an entire line of type at a time, at the expense of a slightly more difficult process for correcting errors (Brewer 79ff).

These machines not only produced metal type but also offered means of efficiently “justifying” a column of type to produce even left and right margins—something that took considerable skill even with hand-set type. The Linotype system was brilliantly simple: wedge-shaped “spacebands” were used to fill inter-word spaces. The operator could literally *squeeze* a line out to full column width. The Monotype system was a little more complicated, with a system of metrics based on an 18-unit width system: each character has a specified width, and the machine counted the units as text was keyed in.³ This is more complex than the Linotype approach, but note how a mathematical system like Monotype’s offers further possibilities for automation and computation. Eventually, the metrics-based approach was adopted by Linotype as well. Both machines (Monotype from the beginning; Linotype later in the 20th century) could also be keyed “remotely” via a punched paper tape interface, separating the compositing keyboard from the type-casting unit and the presses. This separation of functions allowed a division of labour in a typesetting and printing operation, often along gender lines, that had titanic implications for labour relations in the printing industry.⁴

It’s not hard to imagine how a computer could be inserted into this operation: a compositor’s keyed text could be captured, viewed on a screen, corrected, and even saved for future use. The protocols for driving typesetting machines via punched paper tape grew out of teletype and telegraph systems. A standardized code represented each letter of the alphabet, number, and punctuation and could then be represented as a serialized stream of codes, either on punched tape, or over a wire, or indeed in computer memory. These codes are the direct ancestors of the ASCII text standard that dominated computing in the English-speaking world (and, to its detriment, much of the non-English world too) until quite recently.⁵

By the 1960s, the trend towards automation in typesetting was firmly established. Newspapers came first, but the rest of the typesetting world followed suit. In the face of automation and changes in the organization of workers, a typographers’ strike that began in the 1960s ended with the utter demise of the International Typographical Union in the 1980s. By the 1960s too, hot-metal typesetting was itself on its way out, to be replaced by a photographic technique, in which letterforms on photographic negatives were exposed by strobe light onto photographic paper or film. The same automated systems which drove hot metal type casters were used to drive phototypesetters. By the early 1980s, this had given away bit by bit (pun intended) to computer-controlled laser technology, which afforded vastly more control and flexibility in page layout. The technology found in your office laser printer is also used to produce photographic film negatives, which are then exposed onto photosensitive printing plates; the process remains the dominant printing technique today.

By the 1970s computerization had become a dominant theme in printing and typesetting. And yet, despite this revolutionary technological innovation and

enormous changes in business and labour organization, the highly formalist aesthetic traditions of typesetting and the form of the book have stayed remarkably intact. The age of print codified a set of systems and machines for making and thinking about text: a construction based on a formalist understanding of words, assembled into lines, assembled into pages. This model of the text underpins many of our most basic reading, interpretive, and indeed computational approaches today, both analytic and synthetic.

2. *Unix and the text file*

Thomas Kuhn famously argued that scientific progress is not continuous but only advances within paradigms. Eventually, a paradigm's limitations begin to be a problem, and a disruptive phase ensues in which the old paradigm gives way to the new, often taking a generation or more.

In the early 1970s, computing could be described as “pre-paradigmatic”—that is, various systems, strategies, and approaches co-existed, none of them truly dominant. In the 1960s, major efforts in American computer science had attempted to establish such things as interactive computing, time-shared computing resources, and computer networks, via the Pentagon's ARPA funding program. The most famous result of that program is the Internet itself, but another project—a huge consortium endeavour to develop a standardized time-sharing system—is also key: “Multics,” developed by a group that included MIT, General Electric, and AT&T Bell Labs, collapsed under its own weight (as big technology projects are often wont to do) in 1969. In the wake of this, a handful of programmers at Bell Labs, arguably with too much time on their hands, designed a small and simple operating system based largely on the lessons learned from the failed Multics project (Salus 11–12).

They called the system Unix,⁶ as a bit of a joke, and it was intended only for internal use at Bell Labs. The story goes that the two key programmers, Thompson and Ritchie, wanted to play *Spacewar*, an early video game, but needed an operating system in order to repurpose an older computer sitting about unused and that this incidental use was the origin of Unix. In order to justify the time they were spending on Unix, the core programmers promised Bell Labs that they would produce a document-processing package for use by AT&T.

AT&T at this time were legally prevented from entering the computer market, due to anti-trust concerns from the US Department of Justice. As a result, Unix was not marketed but rather spread beyond AT&T via academic channels. Ken Thompson showed Unix at conferences and shared it widely with universities around the world, beginning in 1974. Its simple, modular design caught on, not merely because it worked well but because it was *teachable* (Libes and Ressler, 8). Furthermore, its modularity meant that programmers could extend it, fix it, and help it evolve. By the late 1970s, it had become a dominant system in use in computer science departments around the English-speaking world, and its influence went everywhere. In the early 1980s, the first working implementations of modern

Internet protocols were built and released for Unix, which further cemented Unix's centrality in the emerging online world.

Unix provided a set of concepts that are so essential today that they're hard to think outside: files, filesystems, computing processes, networking. Unix or Unix-derived systems are almost literally everywhere today. Mac OS X is Unix-derived, as is iOS, as is the Android operating system. Microsoft's MS-DOS and Windows were developed as proprietary technology but hugely influenced by Unix. With the rise of the free and open-source software movement in the 1990s, the Linux operating system (written to be independent of Unix from a copyright perspective but following its design principles very closely) became the dominant Unix-like system and now runs most Internet infrastructure. Linux is in most embedded systems, microcontrollers, cars, airplanes, toasters, and whatnot. Unix defines a computing paradigm that we still live within.⁷

In Unix,

- Everything is a file or is at least treated as a file.
- The core file format is simply a sequence of text characters.
- The output of one process (as text) can become the input (as text) of another.
- An interactive “shell” environment (the “command line”) provides a standard interactive textual input and output mechanism.
- System documentation (manuals) are part of the system itself.

The promise made by the original programmers to AT&T back in the early 1970s resulted in a toolkit called the Documentor's Workbench, a set of simple text-processing tools that are still with us, 40-plus years later. The internal documentation system for Unix includes a whole range of functionality, including editing software, spell checkers, grammar checkers, formatters, and indeed typesetting software.

These features make Unix a machine built of text. It is phenomenally useful for text processing, because text processing is at its very heart. Its tools can be used—and combined—to perform a variety of text processing tasks. A great example of this is Regular Expression matching: a late 1970s tool for finding regular patterns in strings of text. Regular Expression matching functions are shot through the Unix toolset, and we encounter them in a vast range of software, from simple text editors to sophisticated layout and text management systems. Regular Expression matching represents one of the basic functions of text in the Unix mindset.

In Unix, a text is a file: a linear sequence of characters and words in a discrete package, with a beginning and an end. Within the file there may be structures which are discernible, but they are not fundamental to the file. Internal structures can be identified by software or interpreted by a reader, but text files are first-class entities in Unix. When we download the text of a book as a file, when we write and edit with text editors or word processors, and when we feed the text into topic modelers and other DH analytical tools, we are typically working in the Unix

mode: a text is a linear sequence of words. Whether we are conscious of working with Unix or not, this computing paradigm has not only provided us with practical tools but a whole mindset and model of the text.

3. Marking up structure

In 1970s-era text processing applications, a major concern was with *delimiters*: how to formally separate the different parts of a text for processing. For example, in preparing a text for typesetting, the formatting codes to direct the typesetting machine would have to be inserted into the text at appropriate points, to distinguish headings, paragraphs, and so on. The tradition of marking up a text for composition is probably almost as old as printing itself; the challenge was to make computer-readable markup in an unambiguous yet unobtrusive way.

Many early markup systems took an approach that can be summed up as: “now do this.” Typically, a specially delimited typesetting instruction would precede some text to be formatted, followed by another instruction and then more text, and so on. In the earliest systems, the markup would literally be raw machine instructions to the typesetting device (in some computer language or other). This, not surprisingly, could be rather unwieldy. The emergent solution was to hide the complexity of the raw instructions in shorthand forms or “macros” that would be relatively unobtrusive. A second problem—that of avoiding dependence on one or other particular typesetting machine—was also solved here, by making the markup shorthand *generic*, as opposed to specific to any one device or vendor. For example, in the old Unix typesetting system “troff,” formatting macros might look like this, where “TI” defines a title and “PP” defines a paragraph:

```
.TI
```

```
On Style
```

```
.PP
```

```
It would hard to find any reputable literary critic
today who would care to be caught defending as an idea
the old antithesis of style versus content ...
```

This “prescriptive” or “procedural” markup—in myriad variations promoted by myriad vendors—worked reasonably well in a linear fashion. It was not, however, particularly good for taking context into consideration (e.g., what does a block quotation look like within a footnote?) nor for search and retrieval applications. Through the 1970s, work at IBM by a team led by Charles Goldfarb and by interested members of the Graphic Communications Association (GCA, a professional association in the printing industry, now *Idealliance*) led to an alternative approach, which became known as Generalized Markup (Goldfarb and Rubinsky). Rather than prescriptive markup (“now do this”), generalized markup was “descriptive markup,” delineating and defining structures within a text by a system of markup

containers. Markup delimiters came in pairs—a start tag and an end tag—and container elements could be nested within one another:

```
<essay>
<title>On Style</title>
<paragraph>It would hard to find any reputable literary
critic today who would care to be caught defending as
an idea the old antithesis of style versus content ...
</paragraph>
...
</essay>
```

Generalized markup was enshrined as an international standard (Standard Generalized Markup Language or SGML, ISO8879:1986), and branches of this tradition were later realized as XML and as the HTML of the World-Wide Web. Digital humanists will also recognize the structure underlying the Text Encoding Initiative (TEI), a “document type” defined in SGML and later XML.

The idea of the Generalized Markup standard was not simply to reduce the chaos of many parallel attempts to solve the same problems but also to define a common paradigm for text processing. Any document prepared using SGML can theoretically be processed by any piece of software that is designed around the SGML specification. Transparency and interoperability were core design goals of Generalized Markup, and it was picked up in earnest in the 1980s by the aerospace industry; the US Department of Defense in 1987 adopted SGML as part of its Continuous Acquisition and Lifecycle Support (CALs) program for all technical documentation. In the same year Coombs, Renear, and DeRose proposed the use of SGML in scholarly text processing—which shortly led to the TEI.

In Generalized Markup (SGML & XML):

- 1 Markup describes the semantic structures of the text
- 2 The text’s structure is separated from its formatting;
- 3 Markup languages can be designed for specific applications;
- 4 Markup tags come in pairs and are containers;
- 5 The structures of the text form a nested hierarchy.

Despite being an international standard, SGML remained a coterie technology—especially in aerospace, medicine, and the digital humanities—until Tim Berners-Lee used SGML to define the HyperText Markup Language for his World-Wide Web project in 1990. Berners-Lee’s simple system spread across the global Internet like wildfire through the 1990s, with the result that we now live in a world where markup is ubiquitous; upwards of a trillion web pages use a generalized markup variant. XML was designed in the late 1990s to be a lighter, web-native version of the SGML standard, still malleable enough to be adapted to industry specific markup languages.

What the markup tradition gives us is an alternative conception of the text. Rather than a linear sequence of characters (as in Unix), the marked-up text is, famously, an *Ordered Hierarchy of Content Objects* (deRose et al.) owing to the hierarchy of nested elements. Processing software can read the file containing marked-up text and parse it into a tree structure in computer memory. A tree structure is a very useful thing for a great range of applications, from search and retrieval to contextual processing, from structural manipulations to transformations into different kinds of documents. It also provides a conceptual structure for thinking about texts, as deRose et al. “OHCO Thesis” argues.⁸

My point in relating the histories of print typesetting, Unix, and Markup is to show how these historical traditions overlap and co-exist in a complex fabric of textual-technical culture. The markup tradition does not replace the Unix tradition any more than the digital replaces print. We use these traditions, strategies, and the tools that embody them, in varying combinations, all the time.

I am less interested in the theoretical superiority of any of these ways of thinking about and working with text—*mediations and remediations all*—than in the ways in which they interweave: where the traditions of typesetting and practical print materialities are both embraced and obviated by turns in the Unix/textfiles approach, in which print culture is radically remediated (or alternatively, betrayed) in the XML/TEI tradition, in the practical hybridity of ordered content hierarchies that are simultaneously plain old linear text files. And at the same time we still think and work in terms of *pages*, which are not well captured in either the Unix approach or the markup tradition but which are fundamental to desktop publishing software. In practice, every act of editing, tagging, formatting, production, and distribution is a kind of generative *deformation* of the text (to use Jerome McGann’s word)—or, better (to use Randall McLeod’s), a *transmission*. In the *Text Processing Techniques & Traditions* course at DHSI, we indulge in textual playfulness, moving back and forth between these mode(l)s, in a demonstration that texts are more than any one thing, greater than any one embodiment, while remaining necessarily embodied.

The “Textual Imaginary”

While various conceptual models of the text coexist, overlap, and perhaps compete for our attentions, they are all grounded in the materialities that give rise to them. A text made of lines and pages is given to us by the technologies of paper, ink, and print. A linear text file made of characters and words in a continuous sequence is given to us by the particular set of decisions about software architecture and indeed user interface metaphors in Unix. And the ordered content hierarchy of an XML parse tree is afforded by not only the constitutional rules of Generalized Markup, but by a toolset populated by parsers, validators, and transformation engines. All have their histories, their design goals, and their cultural trajectories. Humanism inherits them all, often in tangled combinations.

It is not only in the *material* of print and computation in which these models reside. It is also in the *imaginings* of textual scholars, editors, software developers,

and digital humanists who engage with texts in these ways. Tara McPherson's article once again sheds helpful light, in a passage that critiques the structural design of Unix:

In effect the tools chunk computational programs into isolated bits where the programs' operations are meant to be "invisible to the user" and to the other programs in a sequence.

(146)

I believe McPherson is calling out this "invisibility" in a negative sense, which is not the only way it can be read. Mike Gancarz, in his 2003 book *Linux and the Unix Philosophy*, notes that "The creators of the Unix operating system started with a radical concept: They assumed that the user of their software would be computer literate from the start" (xxv). Indeed, this aspect of Unix and the culture that has embraced it for decades is quite rare in popular computing. In contrast to the more visible products of Apple, Microsoft, Adobe, and a host of other companies, Unix makes few allowances for new users. "Read the f***ing manual" is an old adage in the Unix community—possibly replaced today by "Just Google it." And, while the raw elitism and exclusion in such a philosophy have been roundly—and rightly—criticized by many through the years (McPherson), there is an additional facet to this that has been little noted that I would like to call attention to here. Tara McPherson again:

I would argue that to study image, narrative, and visibility will never be enough if we do not engage as well the nonvisual dimensions of code and their organization of the world.

(153)

The nonvisual dimension (that which requires an act of imagination) means that an adept user works in a nonvisual mode. That is, the user must fill in what is absent with her imagination, an abstracted understanding of what is going on in the software. The Unix power user who fluidly interacts via lightning-fast keystrokes while looking at nothing more than a terminal window is the clichéd example. But I would like to go further and suggest that we *all* work in this imagined space when we work with text on computers. For in order to put textual software to work for us, we are required to enter into its world—or better: to internalize that world so that we may imagine, deliberate, plan, and have reasonable expectations of what is going to happen when we tag, process, search-and-replace, or mine text for patterns, topics, stylometric data. We carry, for better or for worse and to a more or less effective extent, a "textual imaginary" that encompasses the text, its structures, and how the software at hand will act upon those structures.

Interestingly, the exception to this is Microsoft Word, which does nothing but show us what the text looks like. As a word "processing" tool is it remarkably one-dimensional. But Microsoft Word, in its quotidian guise at least, is hardly a text

processing tool as digital humanists might understand the term. Microsoft (and Apple, and Adobe) may have made fortunes by giving users a visual representation of their digital work, but when we get out of Word and turn to Oxygen, Sublime Text, NVivo, TAPoR—and certainly when we start working with shell scripts, Pandoc, Python, R, XSLT—we are heavily invested in the textual imaginaries by which these tools make sense.

Lest we be tempted to think of this as a binary between the concrete (visual) and the abstract (imaginary), let me also argue that the Unix command line is of course every bit as concrete as Microsoft Word—and for that matter—every bit as concrete as ink on paper; there is no escaping the materiality (and of course the material history) of our tools and mediations.

Where digital tools become truly effective is where the concrete affordances of the machine (both hardware and software) are married with a powerful⁹ set of imagined constructs in the mind of the user (though “user” is such a terrible word; may we say “reader” or “editor” instead?). The implication here is that the imagined part of computing—for my purposes here, the “textual imaginary”—is at least half of what we mean when we talk about software and digital media.¹⁰

The imaginary is not something we come up with all on our own. It is not a subjective phenomenon; it is cultural and learned; it has a history; it is relational. This is why it is important to attend to the cultural histories of our tools and to not take them simply as given. The imaginaries we inherit with our software are every bit as important as the digital tools themselves. Our effectiveness with them, and our ability to critically interrogate them, depends on the sophistication of our understanding.

Notes

- 1 Gutenberg is a very shady historical character and quite possibly didn't invent any of the things he's been credited with over the centuries. Keith Houston's popular history *The Book: A Cover-to-Cover Exploration of the Most Powerful Object of Our Time* provides a delightful narrative of everything Gutenberg did not actually invent.
- 2 On women in the printing trades, see esp. Baron; Duffy; also www.smu.edu/Bridwell/SpecialCollectionsandArchives/Exhibitions/FiftyWomen/EarlyPrinters/; <https://earlyamericanists.com/2014/10/14/telling-the-story-of-women-in-printing/>; <http://ilove typography.com/2014/10/15/the-first-female-typographer/>
- 3 For detail, see *The Monotype System: A Book for Owners & Operators of Monotypes*. Lanston Monotype Machine Co, 1912. Available at <http://www.circuitousroot.com/artifice/letters/press/comptype/monotype/literature/technical/index.html#monotype-system-1912-google>
- 4 On how gender played into the demise of typographers unions, see Cockburn; Zerker.
- 5 The marvelous website *Circuitous Root* provides a stunning collection of primary sources on the evolution of teletype-driven typesetting, along with myriad other historical offshoots. See <http://www.circuitousroot.com/artifice/telegraphy/tty/codes/index.html>
- 6 As a trademark of AT&T in the 1970s and 1980s, it was UNIX (allcaps); as a vast collection of operating systems, tools, and digital culture, it is now (much more commonly) Unix.
- 7 Forgive me for relegating Steve Jobs to a footnote, but as far as this story is concerned, Jobs and Apple Computer are just that. Despite the enormous contribution Apple has

made to the popularization of computing—not to mention to Desktop Publishing—the Unix tradition of files and text run straight through the Apple era. A host of related companies spun off from Xerox PARC in the 1970s (most notably Adobe Systems) have contributed greatly to the user interface of publishing and typesetting software but without affecting much of the underlying structure.

- 8 The “OHCO thesis is by now the subject of an enormous debate about the nature of the text. From a textual studies perspective, the OHCO thesis lost in theory but won in practice” (Galey in McCarty 112).
- 9 The other senses of “power” are not lost on me here. McPherson’s article is straightforwardly about social justice and where that may or may not fit neatly in our relationship to our machines. Where issues of power and social justice connect is, I think, precisely in these shared imaginaries: of what our machines can do and what we expect of them.
- 10 When we get into neural networks and machine learning, we leave the (human) imaginary behind, and I begin to be worried.

Works Cited

- Baron, Ava, editor. *Work Engendered: Toward a New History of America Labor*. Ithaca: Cornell UP, 1991. 385 pp. Ithaca: Cornell University Press, 1991. Print.
- Barthes, Roland. “From Work to Text.” *The Rustle of Language*. Trans. Richard Howard. Berkeley and Los Angeles: University of California Press, 1971, pp. 56–64. Print.
- Brewer, Roy. *An Approach to Print: A Basic Guide to the Printing Processes*. London: Blandford P, 1971. Print.
- Clod, Random, and Random Clod. “Information on Information.” *Text*, vol. 5, 1991, pp. 241–281. Print.
- Cockburn, Cynthia. *Brothers: Male Dominance and Technological Change*. Pluto P, 1991. Print.
- Coombs, James H., et al. “Markup Systems and the Future of Scholarly Text Processing.” *Communications of the ACM*, vol. 30, no. 11, 1987, pp. 933–947. ACM Digital Library. Web.
- DeRose, Steven J. et al. “What Is Text, Really?” *ACM SIGDOC Asterisk Journal of Computer Documentation*, vol. 21, no. 3, 1997, pp. 1–24. Print.
- Duffy, Patrick. *The Skilled Compositor, 1850–1914: An Aristocrat Among Working Men*. Routledge, 2017. Print.
- Edwards, Paul N. *The Closed World: Computers and the Politics of Discourse in Cold War America*. Cambridge: MIT Press, 1996. Print.
- Galey, Alan. “Religious Issues: Form and Content.” *Text and Genre in Reconstruction: Effects of Digitalization on Ideas, Behaviours, Products and Institutions*, edited by Willard McCarty. Open Book Publishers, 2010, pp. 110–114. Print.
- Gancarz, Mike. *Linux and the Unix Philosophy*. Digital Press, 2003. Print.
- Goldfarb, Charles, and Yuri Rubinsky. *The SGML Handbook*. Oxford: Oxford University Press, 1991. Print.
- Haraway, Donna Jeanne. *The Haraway Reader*. Psychology P, 2004. Print.
- Houston, Keith. *The Book: A Cover-to-Cover Exploration of the Most Powerful Object of Our Time*. WW Norton, 2016. Web. 30 Nov. 2016.
- Kuhn, Thomas S. *The Structure of Scientific Revolutions*. 3rd edition. Chicago: University of Chicago Press, 1996. Print.
- Libes, Don, and Sandy Ressler. *Life With UNIX: A Guide for Everyone*. Englewood Cliffs, NJ: Prentice-Hall, 1989. Print.
- Lundblad, Kristina. *Bound to Be Modern: Publisher’s Cloth Bindings and the Material Culture of the Book 1840–1914*. New Castle DE: Oak Knoll Press, 2015. Print.
- Maxwell, John. “Tracing the Dynabook: A Study of Technocultural Transformations.” Doctoral Dissertation. U British Columbia, 2006. Web. <http://tkbr.publishing.sfu.ca/dynabook/>.

- McGann, Jerome. *Radiant Textuality: Literary Studies After the World Wide Web*. New York: Palgrave Macmillan, 2001. Print.
- McLuhan, Marshall. *The Gutenberg Galaxy*. Toronto: University of Toronto Press, 1962. Print.
- McPherson, Tara. "Why Are the Digital Humanities So White? Or Thinking the Histories of Race and Computation." *Debates in the Digital Humanities*. Minneapolis: University of Minnesota Press, 2012. Web. 19 June 2017.
- Ong, Walter J. *Orality and Literacy: The Technologizing of the Word*. London and New York: Routledge, 1995. Print.
- Salus, Peter H. *A Quarter Century of Unix*. Reading, MA: Addison-Wesley, 1994. Print.
- Smeijers, Fred. *Counterpunch: Making Type in the Sixteenth Century, Designing Typefaces Now*, edited by Robin Kinross. Hyphen, 1996. Print.
- Stephenson, Neal. *In the Beginning . . . Was the Command Line*. New York: William Morrow Paperbacks, 1999. Print. http://project.cyberpunk.ru/lib/in_the_beginning_was_the_command_line/.
- Zerker, Sally F. *The Rise and Fall of the Toronto Typographical Union 1832–1972: A Case of Foreign Domination*. Toronto: University of Toronto Press, 1982. Print.